EduAirControl — Software Implementation

> Procedure for preparing, building, configuring, deploying and verifying the system

Servicio Nacional de Aprendizaje · Análisis y Desarrollo de Software – 3145556
Integrantes: María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas
Instructores: Todos los instructores del programa
Versión 1.0 · 9 de octubre de 2026 · 2025-2026

Tabla de contenido

- 1. Introducción
- 2. Objetivo
- 3. Alcance y criterios de lectura
- 4. Requisitos de implementación
  - 4.1 Herramientas
  - 4.2 Requisitos de hardware
  - 4.3 Puertos
- 5. Organización del código
- 6. Configuración de variables de entorno
- 7. Implementación local con Docker Compose
  - 7.1 Preparación
  - 7.2 Construcción y arranque
  - 7.3 Espera y comprobación de servicios
  - 7.4 Stack de observabilidad
- 8. Implementación del frontend web
- 9. Implementación de la aplicación móvil
- 10. Implementación de base de datos y migraciones
- 11. Implementación de la integración IoT
- 12. Integración continua y construcción de imágenes

- 13. Despliegue en Kubernetes
- 14. Verificación y pruebas posteriores
- 15. Despliegue, reversión y recuperación
- 16. Seguridad de la implementación
- 17. Estado de implementación y pendientes
- 18. Entregables y evidencias
- 19. Glosario
- 20. Referencias internas

---

1. Introducción

This document describes how to deploy EduAirControl from the delivered repository. The system integrates a web application, a mobile application, an API Gateway, Spring Boot services, PostgreSQL databases, Redis, RabbitMQ, observability tools and an IoT prototype based on ESP32.

The guide is written for a development, integration, demonstration and deployment-preparation environment. The existence of a Dockerfile, a Kubernetes manifest or an observability configuration means the mechanism is prepared or documented; it does not automatically mean it has been validated for production.

2. Objetivo

Define a reproducible procedure to:

- obtain and organize the source code;
- prepare tools and environment variables;
- build and run the services;
- create and update databases using Liquibase;
- deploy the web frontend and the mobile application;
- integrate the ESP32 prototype with DHT11, KY-038 and MQ-135;
- run health, smoke and acceptance checks;
- publish images via continuous integration;
- prepare a controlled deployment on Kubernetes;
- perform a rollback or recovery when necessary.

3. Alcance y criterios de lectura

Incluido

- Local deployment with `docker-compose.yml` in `microservicios/ea-control-infra`.
- Build of `api-gateway`, `ms-security`, `ms-classroom-management`, `ms-sensor-management`, `ms-environment-monitoring`, `ms-user-management` and `ms-user-experience`.
- Independent PostgreSQL databases per service in the local stack.
- Redis for session, blacklist and rate limiting.
- RabbitMQ for events and asynchronous communication.
- React/Vite web frontend served by Nginx in a Docker image.
- Expo/React Native mobile application with BLE provisioning.
- Deployment manifests for Kubernetes.
- CI for Java services using GitHub Actions.
- Prototype tests with DHT11 and KY-038.

Not confirmed as production

- Final firmware that sends ESP32 readings to the API.
- Metrological calibration of DHT11, KY-038 or MQ-135.
- Final production strategy, cloud provider, domain and certificates.
- Test coverage measured with JaCoCo.
- Complete observability connected to effective metrics of each service.
- Restore of backups tested in an operational environment.
- Final separation of all data and contracts between services.

> Rule: example values, local passwords, `example.com` domains and demo keys must be replaced before any institutional or production use.

4. Requisitos de implementación

4.1 Herramientas

Herramienta	Versión o condición	Uso
Git	2.40 or higher recommended	Obtain code and manage versions
Docker Desktop	Docker Engine 24 or higher recommended	Local containers
Docker Compose	2.20 or higher recommended	Orchestrate the stack
Java	JDK 17	Build and test Spring Boot services
Maven Wrapper	`./mvnw` or `mvnw.cmd`	Compile each service
Node.js y npm	Node 20 recommended for the frontend Docker; compatible with Expo according to the project	Web and mobile frontend
Android Studio	For native Android execution	SDK, emulator and `expo run:android`
Arduino IDE	For ESP32 testing	Upload test firmware
`curl` y `jq`	Available on the machine	Health checks and smoke test
Kubernetes y kubectl	Only for K8s deployment	Apply manifests and inspect pods

4.2 Requisitos de hardware

- Computer with a recommended minimum of 8 GB of RAM.
- Space for Docker images, dependencies, builds and PostgreSQL volumes.
- ESP32.

- DHT11 for temperature and humidity.
- KY-038 for relative sound level.
- MQ-135 for relative gas/air quality changes.
- Breadboard, jumpers and safe power supply.
- Android/iOS phone or emulator to test the mobile app.

4.3 Puertos

Servicio	Puerto publicado
API Gateway	8080
`ms-security`	18081 on host; 8081 inside Docker
`ms-classroom-management`	3002
`ms-environment-monitoring`	3003
`ms-sensor-management`	3004
`ms-user-experience`	3006
`ms-user-management`	3007
PostgreSQL security	5437
PostgreSQL classroom	5433
PostgreSQL sensors	5434
PostgreSQL monitoring	5435
PostgreSQL user management	5438
PostgreSQL user experience	5439
Redis	6379
RabbitMQ AMQP	5672
RabbitMQ Management	15672
Prometheus	9090
Grafana	3001
OpenTelemetry OTLP	4317 / 4318

Servicio	Puerto publicado
Frontend web Docker	80 inside the container; publish on 3000 or 5173 depending on environment

Before starting, check for port conflicts. On Windows you can use `netstat -ano`; on Linux/macOS, `lsof -i :<puerto>`.

5. Organización del código

The delivery contains two main areas:

```
proyecto/
├── Docs-ea-control/                        # Documentación de ingeniería
├── ea-control-frontend/
│   └── apps/
│       ├── web/                            # React + Vite + Nginx
│       └── mobile/                          # Expo + React Native
├── microservicios/
│   ├── ea-control-api-gateway/
│   ├── ea-control-ms-security/
│   ├── ea-control-ms-classroom-management/
│   ├── ea-control-ms-sensor-management/
│   ├── ms-environment-monitoring/
│   ├── ea-control-ms-user-management/
│   ├── -ms-user-experience/
│   └── ea-control-infra/                    # Compose, K8s, CI y scripts
└── manuales/                                # Manuales de usuario, técnic
```

The central infrastructure is in `microservicios/ea-control-infra`:

- `docker-compose.yml`: main stack.
- `docker-compose.observability.yml`: Prometheus, Grafana and OpenTelemetry Collector.
- `scripts/wait-for-services.sh`: wait for health endpoints.
- `scripts/smoke.sh`: registration, login and logout test.
- `deploy/k8s/`: namespace, apps, databases, Redis, RabbitMQ, secrets and ingress.
- `.github/workflows/build-services.yml`: tests, build and image publishing.

6. Configuración de variables de entorno

Create environment files from the examples, without versioning real values:

```
cd proyecto/Docs-ea-control
cp .env.example .env

cd ../ea-control-frontend/apps/web
cp .env.example .env

cd ../mobile/EduAirControlMobile/Front-End
cp .env.example .env
```

In the infrastructure stack, Compose accepts variables such as:

Variable	Uso local
`GATEWAY_PORT`	API Gateway port, default 8080
`CORS_ALLOWED_ORIGINS`	Allowed frontend origins
`SECURITY_DB`, `SECURITY_USER`, `SECURITY_PASSWORD`	Security database and credentials
`CLASSROOM_DB`, `SENSOR_DB`, `MONITORING_DB`	Domain databases
`JWT_SECRET`	Validation for services that still require it
`RABBITMQ_USER`, `RABBITMQ_PASSWORD`	Broker credentials
`REDIS_PORT`	Redis port
`REGISTRY`, `TAG`	Image registry and tag
`GRAFANA_USER`, `GRAFANA_PASSWORD`	Local Grafana access

The frontend uses `VITE_API_URL=http://localhost:8080`. The mobile app accepts `EXPO_PUBLIC_API_URL`; on a physical phone it must point to the machine LAN IP, for example `http://192.168.1.20:8080`.

To generate a temporary local key you can use:

```
openssl rand -base64 32
```

Do not use the credentials `guest/guest`, `change-me-in-vault` or default values in production.

7. Implementación local con Docker Compose

7.1 Preparación

1. Install Docker Desktop and verify the Linux engine is active.
2. Open a terminal at the project root.
3. Verify that the service projects are located alongside `ea-control-infra`.
4. Create or review the stack `.env` file.
5. Confirm that production keys, domains and passwords are not in the repository.

7.2 Construcción y arranque

From the infrastructure:

```
cd proyecto/microservicios/ea-control-infra
docker compose build
docker compose up -d
```

To build and start in a single step:

```
docker compose up -d --build
```

The stack builds the Spring Boot services and pulls PostgreSQL 15, Redis 7 and RabbitMQ 3.13 Management. Each application service depends on its primary resources and declares a `healthcheck`.

Check status:

```
docker compose ps
docker compose logs -f api-gateway
docker compose logs -f ms-security
```

7.3 Espera y comprobación de servicios

The available script waits for the gateway, security, classrooms and sensors health endpoints:

```
chmod +x scripts/wait-for-services.sh scripts/smoke.sh
./scripts/wait-for-services.sh
```

The maximum timeout can be modified:

```
TIMEOUT=300 ./scripts/wait-for-services.sh
```

The smoke test requires `jq` and should be run only against a development database, because it creates a test user:

```
./scripts/smoke.sh
```

Minimal manual checks:

```
curl -fsS http://localhost:8080/health
curl -fsS http://localhost:8081/api/v1/auth/health
curl -fsS http://localhost:3002/health
curl -fsS http://localhost:3004/health
```

7.4 Stack de observabilidad

To start the main stack and observability:

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.observability.yml \
  up -d --build
```

Local access:

- Prometheus: `http://localhost:9090`.
- Grafana: `http://localhost:3001`.
- OTLP HTTP: `http://localhost:4318`.

The configuration is prepared in `observability/prometheus.yml` and `observability/otel-collector.yaml`. The availability of effective metrics per service must be verified before declaring full observability.

To stop without removing volumes:

```
docker compose down
```

To remove containers and local data — destructive action —:

```
docker compose down -v
```

8. Implementación del frontend web

Ejecución en desarrollo

```
cd proyecto/ea-control-frontend/apps/web
npm install
npm run dev
```

Vite normally serves the frontend at `http://localhost:5173`. The `VITE_API_URL` variable must point to the gateway.

Construcción de producción

```
npm run lint
npm run build
npm run preview
```

The Docker image uses two stages: Node 20 builds the app and Nginx serves `dist`. To build it manually:

```
docker build \
  --build-arg VITE_API_URL=http://localhost:8080 \
  -t eduaircontrol-frontend:local .
```

The Nginx configuration uses `try_files` so React routes work on page refresh. The image internal port is 80; the external port is defined in the consuming deployment.

9. Implementación de la aplicación móvil

The application is located at `ea-control-frontend/apps/mobile/EduAirControlMobile/Front-End`.

```
cd proyecto/ea-control-frontend/apps/mobile/EduAirControlMobile/Front-End
npm install
npm start
```

Available commands:

```
npm run android
npm run ios
npm run prebuild
npm run check:android
npm run lint
```

The app uses Expo 54.0.37, React Native 0.81.5 and React 19.1.0. For a physical phone:

```
EXPO_PUBLIC_API_URL=http://192.168.1.X:8080 npx expo start
```

On the Android emulator, `10.0.2.2` represents the host machine. BLE provisioning looks for the device `ESP32_Config`; Android 12 or higher requires `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` permissions.

10. Implementación de base de datos y migraciones

The main Compose brings up a PostgreSQL database per domain:

- `eduaircontrol_security`.
- `eduaircontrol_classrooms`.
- `eduaircontrol_sensors`.
- `eduaircontrol_monitoring`.
- `eduaircontrol_user_management`.
- `eduaircontrol_user_experience`.

Each service runs its Liquibase changelogs on startup. Changes are located in each service's `src/main/resources/db/changelog`. Hibernate uses schema validation; tables should not be created or modified manually to replace migrations.

Recommended procedure:

1. Start databases first and wait for their `healthy` status.
2. Start application services.
3. Review Liquibase logs.
4. Confirm there are no checksum errors or unapplied changes.

5. Run a read/write test for each domain.
6. Backup before structural changes.

Persistent volumes are `security_data`, `classroom_data`, `sensor_data`, `monitoring_data`, `user_management_data`, `user_experience_data` and `rabbitmq_data`. Do not run `docker compose down -v` if data must be preserved.

11. Implementación de la integración IoT

11.1 Componentes

Componente	Función
ESP32	Controller, reading and Wi-Fi/BLE communication
DHT11	Air temperature and humidity
KY-038	Relative sound level via its analog output
MQ-135	Relative gas/air quality change detection
Protoboard y jumpers	Prototype assembly

The provided test programs document:

- DHT11 DATA on GPIO 5/D5.
- KY-038 AO on GPIO 34/D34.
- Serial monitor at 115200 baud.

11.2 Secuencia de implementación

1. Assemble the ESP32 and sensors on the breadboard.
2. Verify common ground, power and continuity.
3. Upload and run the DHT11 test first.
4. Upload and run the KY-038 test.
5. Evaluate the MQ-135 only with proper voltage adaptation.
6. Define the single firmware and the real ingestion contract.
7. Register the device in the sensor service.

8. Provision Wi-Fi via BLE from the mobile app if the firmware supports it.
9. Send a test measurement with an individual device credential.
10. Verify the measurement is validated, stored and visible on the dashboard.

> The DHT11 and KY-038 have test programs. The final firmware that connects the device to the API, the definitive MQ-135 connection, battery voltage and calibration remain pending confirmation. The KY-038 must not be presented as a calibrated sound level meter and the MQ-135 must not be presented as a precise CO_2 ppm meter.

12. Integración continua y construcción de imágenes

The workflow `microservicios/ea-control-infra/.github/workflows/build-services.yml` runs on `push` to `main`, pull requests to `main` and manual execution.

The current matrix includes six services:

- `api-gateway`.
- `ms-security`.
- `ms-classroom-management`.
- `ms-sensor-management`.
- `ms-user-management`.
- `ms-environment-monitoring`.

Each service configures JDK 17, Maven cache and runs `./mvnw -B verify`. On a `push` to `main`, the workflow builds and publishes images to GHCR with the tags `latest` and the commit SHA.

> `ms-user-experience` exists in Compose and in the code, but does not appear in the current workflow matrix. Before declaring CI complete, that service must be included and verified.

13. Despliegue en Kubernetes

Manifests are located in `microservicios/ea-control-infra/deploy/k8s`.

Preparación

1. Have a Kubernetes cluster and an Nginx Ingress Controller.

2. Create the EduAirControl namespace.
3. Configure real secrets outside the repository.
4. Publish images to a registry accessible by the cluster.
5. Replace example tags and the Ingress domain.
6. Configure TLS via cert-manager or the institutional mechanism.

Aplicación de manifiestos

```
cd proyecto/microservicios/ea-control-infra
kubectl apply -f deploy/k8s/namespace.yaml
kubectl apply -f deploy/k8s/secrets.example.yaml
kubectl apply -f deploy/k8s/configmap.yaml
kubectl apply -f deploy/k8s/postgres.yaml
kubectl apply -f deploy/k8s/redis.yaml
kubectl apply -f deploy/k8s/rabbitmq.yaml
kubectl apply -f deploy/k8s/apps.yaml
kubectl apply -f deploy/k8s/ingress.yaml
```

Verify the deployment:

```
kubectl get pods -n eduaircontrol
kubectl get svc -n eduaircontrol
kubectl get ingress -n eduaircontrol
kubectl rollout status deployment/api-gateway -n eduaircontrol
kubectl logs deployment/api-gateway -n eduaircontrol
```

The manifests include replicas, `readinessProbe`, `livenessProbe`, resource requests and limits. Do not use `secrets.example.yaml` with real passwords. The K8s configuration must be reviewed to confirm that all services present in Compose are also deployed, especially `ms-user-experience`.

14. Verificación y pruebas posteriores

The deployment is considered technically acceptable when at minimum the following are met:

Verificación	Criterio
Contenedores	All required services are `running` and their health checks respond
Gateway	`GET /health` responds successfully
Seguridad	Registration, login, renewal and logout work in the test environment
Gateway	Routes are routed to the correct service
Base de datos	Liquibase completes without errors and data persists after restart
Eventos	RabbitMQ receives and delivers events without duplicate effects
Frontend	`npm run lint` and `npm run build` complete successfully
Móvil	Expo starts and can communicate with the gateway
IoT	DHT11 and KY-038 deliver test values; ingestion firmware is validated separately
Smoke test	`scripts/smoke.sh` finishes without error on a development database
CI	All services included in the matrix run `mvnw verify`
Kubernetes	Pods ready, probes correct and Ingress accessible in a controlled environment

Coverage, availability, performance and recovery metrics should not be declared met without corresponding evidence.

15. Despliegue, reversión y recuperación

Detención segura

```
docker compose stop
```

Reinicio de un servicio

```
docker compose restart ms-environment-monitoring
```

Reconstrucción después de un cambio

```
docker compose up -d --build ms-environment-monitoring
```

Reversión en Kubernetes

```
kubectl rollout undo deployment/api-gateway -n eduaircontrol
kubectl rollout status deployment/api-gateway -n eduaircontrol
```

Before a database migration:

1. stop or isolate write traffic;
2. create a backup of the affected databases;
3. run the migration in a test environment;
4. verify counts, indexes and constraints;
5. deploy the compatible code;
6. keep a rollback plan.

Rolling back an incompatible migration should not be executed automatically; it must be approved by the technical owner after evaluating data loss or transformation.

16. Seguridad de la implementación

- Do not publish `.env` files, private keys, tokens, passwords or Kubernetes secrets.
- Replace demonstration RSA keys and rotate them upon any exposure.
- Use `CORS_ALLOWED_ORIGINS` with explicit origins.

- Keep internal ports on the private network when possible.
- Apply HTTPS for external access.
- Use an individual credential per IoT device, with rotation and revocation.
- Validate JWT signature in each owning service, not only at the gateway.
- Restrict metrics endpoints to the internal network.
- Review Docker images and dependencies before release.
- Do not use the values `guest`, `dev` or `change-me-in-vault` in production.

17. Estado de implementación y pendientes

Área	Estado identificado	Acción pendiente
Código de microservicios	Available in the ZIP	Confirm versions and canonical repositories
Docker Compose	Prepared for the full local stack	Run a clean test and document results
PostgreSQL/Liquibase	Changelogs per service available	Test restore and backward migrations
Redis/RabbitMQ	Configured in Compose	Validate resilience and duplicates
Frontend web	Code, Dockerfile and Nginx available	Confirm final external port
Aplicación móvil	Expo, BLE and services available	Validate on physical Android/iOS device
CI	Workflow for six services	Include `ms-user-experience` and review contracts
Kubernetes	Manifests available	Review images, secrets, domain, TLS and missing services
Observabilidad	Compose and configuration files available	Confirm real metrics, dashboards and alerts
DHT11/KY-038	Test programs available	Integrate final firmware and validate sensor → API chain
MQ-135	Component contemplated	Confirm divider, power, calibration and final use
Seguridad	Controls documented; findings open in quality	Close AC-01, AC-02 and AC-03 before production

18. Entregables y evidencias

To close a deployment it is recommended to keep:

- output of `docker compose ps`;
- startup logs without Liquibase errors;
- health endpoint responses;
- smoke test report;
- result of `npm run lint` and `npm run build`;
- result of `./mvnw verify` per service;
- evidence of login and environment query;
- evidence of one DHT11 reading and one KY-038 reading;
- evidence of the API receiving the IoT measurement, when firmware is ready;
- manifests and output of `kubectl get pods` in K8s;
- backup and test restore record;
- version, commit, date, responsible and environment of each delivery.

19. Glosario

Término	Definición
API Gateway	Entry point that routes requests to services.
CI/CD	Continuous integration and delivery/deployment.
Docker Compose	Tool to run multiple containers in coordination.
Health check	Check that indicates whether a service is available.
Ingress	Kubernetes HTTP/HTTPS entry to services.
Liquibase	Tool for versioning and executing database changes.
RabbitMQ	Messaging broker for asynchronous events.
Redis	In-memory store used for state and cross-cutting capabilities.
Smoke test	Brief test of essential flows after deployment.
BLE	Bluetooth Low Energy used to configure the ESP32.
DHT11	Low-cost temperature and relative humidity sensor.
KY-038	Sound detection module with relative output; not a calibrated sound level meter.
MQ-135	Gas sensor that requires calibration and voltage adaptation.
RTO/RPO	Recovery time objective and maximum acceptable data loss.

20. Referencias internas

- `Docs-ea-control/10-devops/local-setup.md` — local preparation.
- `Docs-ea-control/10-devops/environments.md` — environments and documented pipeline.
- `Docs-ea-control/05-architecture/overview.md` — architecture and technologies.
- `Docs-ea-control/05-architecture/decisions/records/ADR-010-containerization-strategy.md` — containerization strategy.

- `Docs-ea-control/05-architecture/decisions/records/ADR-011-production-deployment-strategy.md` — production deployment strategy.
- `microservicios/ea-control-infra/docker-compose.yml` — local stack.
- `microservicios/ea-control-infra/docker-compose.observability.yml` — local observability.
- `microservicios/ea-control-infra/scripts/wait-for-services.sh` — service wait script.
- `microservicios/ea-control-infra/scripts/smoke.sh` — smoke test.
- `microservicios/ea-control-infra/.github/workflows/build-services.yml` — continuous integration.
- `microservicios/ea-control-infra/deploy/k8s/` — Kubernetes manifests.
- `08_Calidad_Mejoras` — corrective, preventive and improvement actions.
- `04_Propuesta_Tecnica` — architecture, risks and technical decisions.